__/__/____

## Python:

Python is a high-level, interpreted, general-Purpose programming language.

- It was created by Guido Van Rossum and first released in 1991
- Python is designed to be easy to read and Simple to use.

## Features of Python:

- Easy to learn and use:
  Simple Syntax Similar to english
- High-level language
  No need to manage memory manually:
- Interpreted language
  code is executed line by line.
- Objectoriented
  It Support classes and Objects.
- Portable
  Runs on different Platforms
- Extensive Standard library
  Large collection of modules and functions.
- Open Source
  Free to use and distribute
- Dynamically typed
  No need to declare variable types.

__/__/____

## First Python Program:

```
# This is a Simple Python Program
print("Hello, World!")    # prints text on the screen

# Output : Hello, World!
```

## * Variables in Python:

- It is a named memory location that holds Single valued data
- It is a container that holds value.

## Basic rules for python variable:

* A variable can always starts with underscore or a character ex: _name & name
* A variable does not contain a special character except underscore ex: +, -, *, %, $.
* A variable cannot Start with digits.
* It contains alpha numeric charcters.
* A variable can not contains keyword.
  ex: for, is, if.
* A variable can't contains a space
  ex: Number of blades → NumberOfBlades

Camel convension
Camel concadenation

__/__/____

* Reserved keywords in Python

| | | | | | |
|---|---|---|---|---|---|
| False | None | True | and | as | assert |
| break | continue | def | del | elif | else |
| except | finally | for | from | global | if |
| import | in | is | lambda | not | nonlocal |
| or | pass | raise | return | try | while |
| with | yield. | | | | |

# Program to demonstrate Variables in Python.

```
age = 25                    # integer variable
height = 5.9                # float variable
name = "Pooja"              # String variable
is_Student = True           # boolean variable
print ("Name : ", name)
print ("Age :", age)
print ("Height :", height)
print (" Student :", is_Student)
```

output:

```
Name : Pooja
Age : 25
Height : 5.9
Student : True
```

## Data Types in Python:

Python has the following buit-in-data types

1. Numeric Types
2. Sequence Types
3. Set Types
4. Mapping Types
5. Boolean Type
6. None Type.

### 1 Numeric Type:

Used to store numbers. Python has three numeric types.

- int - Integer number. Ex: 10, -25, 0 (4 byte)
- float - Decimal numer. Ex - 3.14, -0.001, 2.0 (4byte)
- complex - complex number. Ex: 2+3j

### 2. Sequence Type:

Used to store a collection of items in an ordered manner.

- Str: String (Sequnce of character.) immutable
- list: list (ordered, mutable collection)
- tuple: Tuple (ordered, immutable collection)

once declared value we can able to modify

Can not modify

__/__/____

3. **Set types :**

- used to store unique items in an unordered manner.

  - Set : Set → unordered collection of unique items
  - frozen set - Immutable version of Set

Ex: `s = {10, 20, 30, 10}` # duplicates are remove

`print (s)`

#output : {10, 20, 30}

4. **Mapping Type (Dictionary) - Dictionary**

- used to store data in key-value pairs.

  - dict - Dictionary (mutable collection of key-value pairs)

Ex: `student = { "name" : "Sagar", "age": 25}`

`print (Student ["name"])`

output = Sagar # value can be changed

5. **Boolean Type :**

- used to represent True or False values.

  - bool - Boolean Type

Ex: `a = True`

`b = False`

`print (type(a))`

True → 1

False → 0

# output : <class 'bool'>

__/__/____

6. None Type :

- used to represent the absence of value

• None Type : None represents null value

Ex: x = None

print (x)

Print (type (x))

# o/p : None

# o/p = <class 'None Type'>

NOTE

None is not the same as 0. False or empty String. It is special value

## Operater :

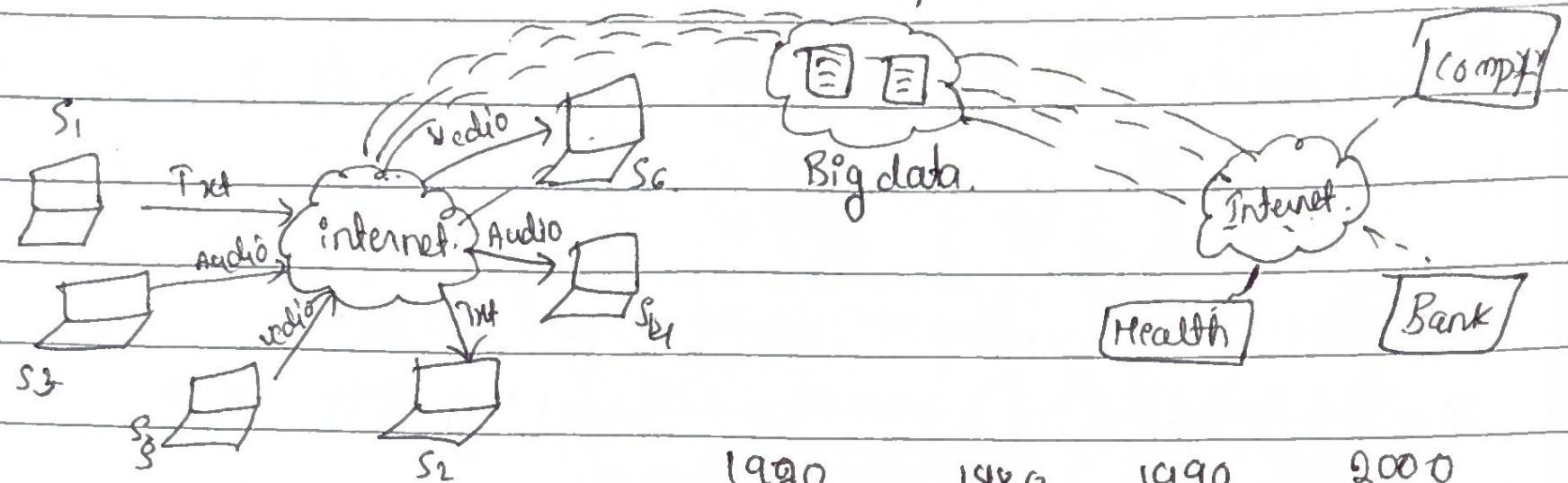
Operation: It is task or an action that is perform on values based on the operators used.

Operand - values on which the operations are performed.

Operator - Special symbols or keywords that includes in built functionality based on which the task is performed on values.

24/07/2026

# Full Stack Developer.

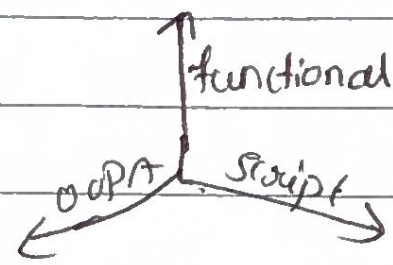



1989 - 1991
P1 → 1994
P2 → 2000
P3 → 2008

| 1990 | 1980 | 1990 | 2000 |
|---|---|---|---|
| C | C++ | JAVA | Perl |
| fundional. | OOPA | OOPA | Script |

latest version P3.14.2

3Dimensional.




## Syllabus:

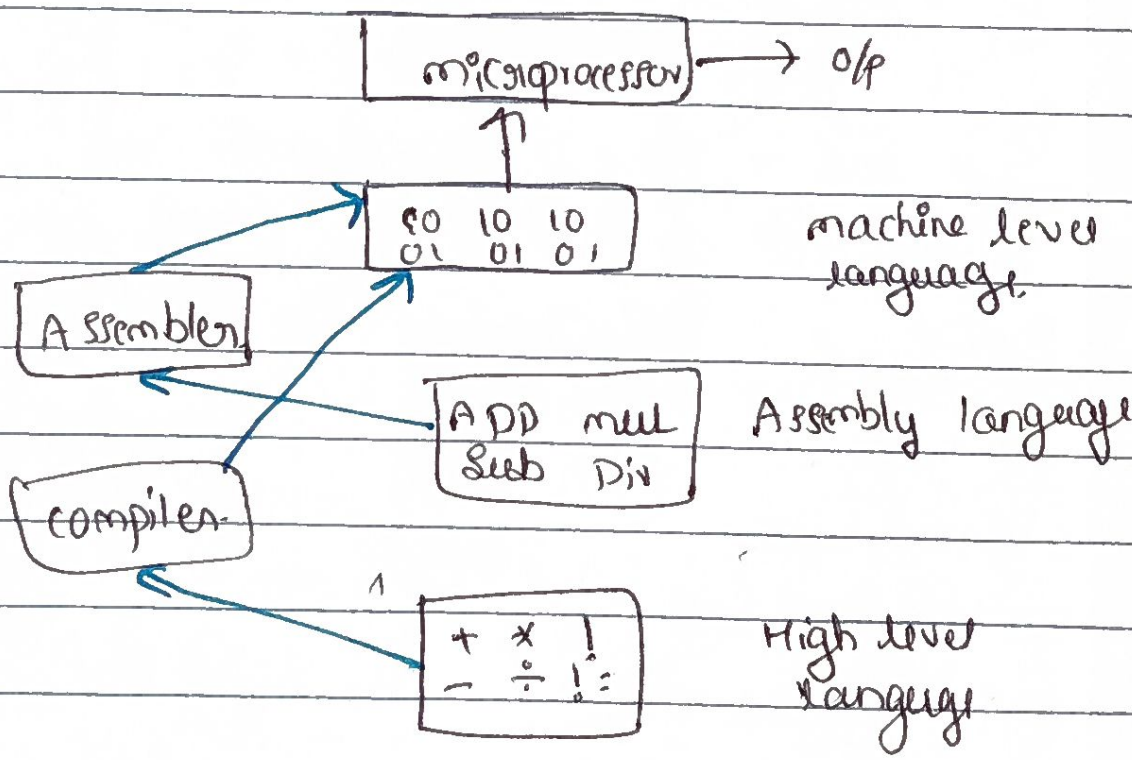
* Basic &
* Numerical Program
* Recursion
* Patterns.
* Arrays
* Strings

## Software.

- Python.org
  - → P3.14.2
- Pycharm.
- Visual Studio code.

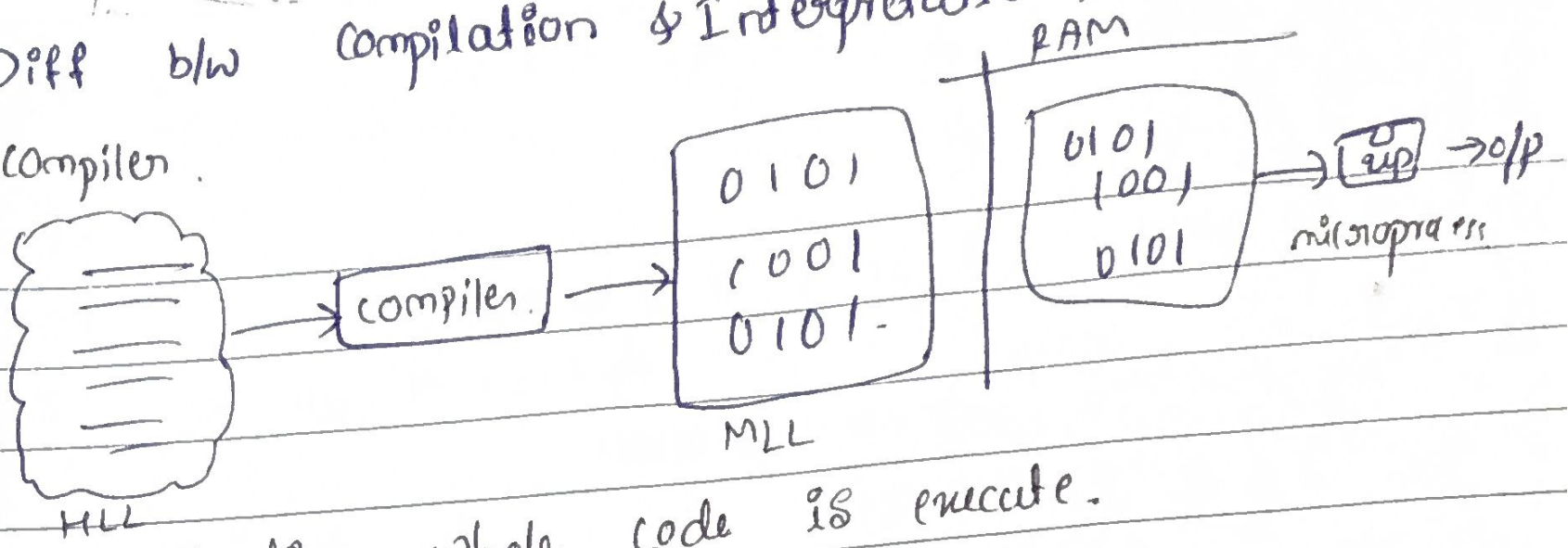
## Introduction:

1950's
↓
Semi-conductor
↓
Transistor.



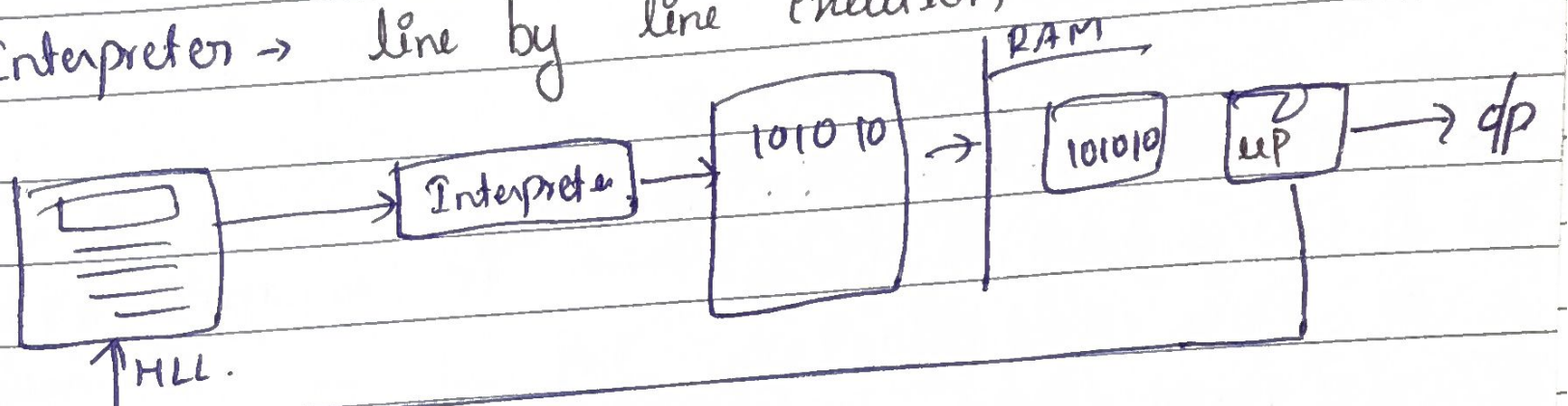

Hackerrank
Hacker earth
leet code
greekforgeeks.

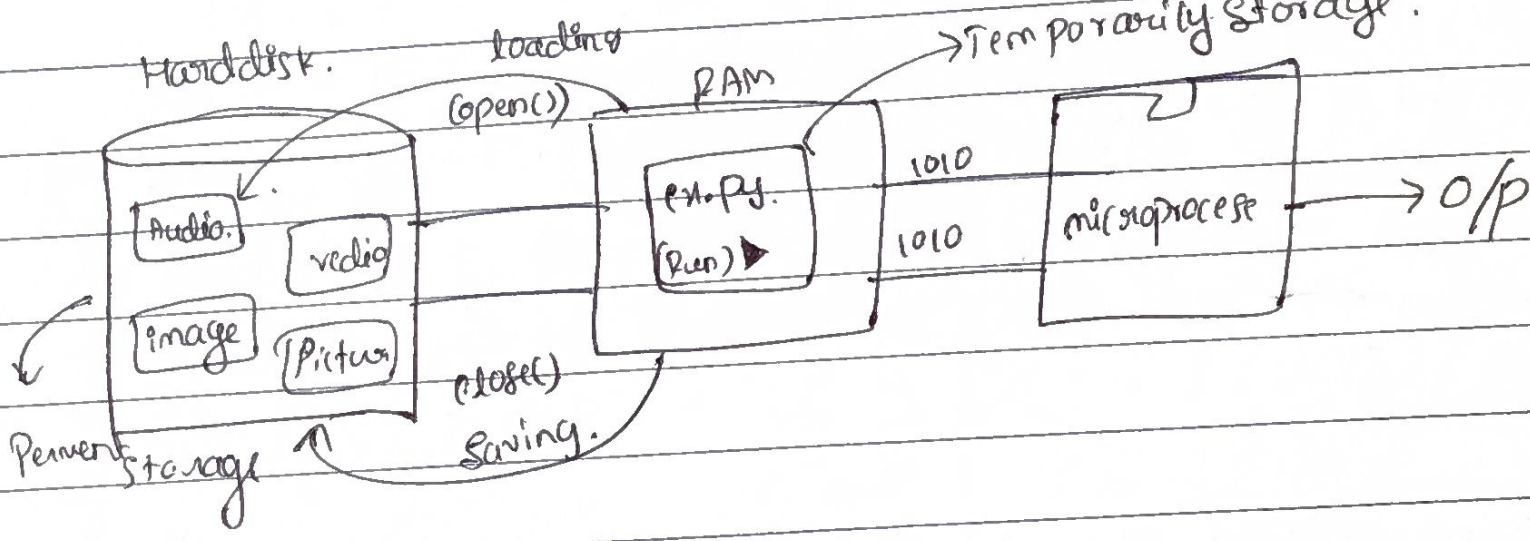
# Diff b/w Compilation & Interpretation.




→ at time whole code is execute.

Interpreter → line by line execution




Python is platform independent.

PVM - Python Virtual Machine. — Heart of Python.

↳ when PVM install in compilation & execution then it will independent platform.

- By default it will Store in String format

  a = int (input ("Enter a value:")

- concatination - add or different add strings.

→ Conditional Statement (if, else, elsif)

22/07/2026 # Looping Statements → for, while.

1) for

a) without range: for i in elements
:
Block of code
#remain Statements.

Ex: for i in 1,2,3,4:
Print(i)
o/p 1 2 3 4.

Str = "pooja"
for i in Str:
Print(i)
o/p: p o o j a.

## 2) Range function in for loops

1) Range (Stop)
2) range (Start, Stop)
3) range (Start, stop, Step)

→ for var_name in range (Stop_value):
:
logic
#remaining Statement.

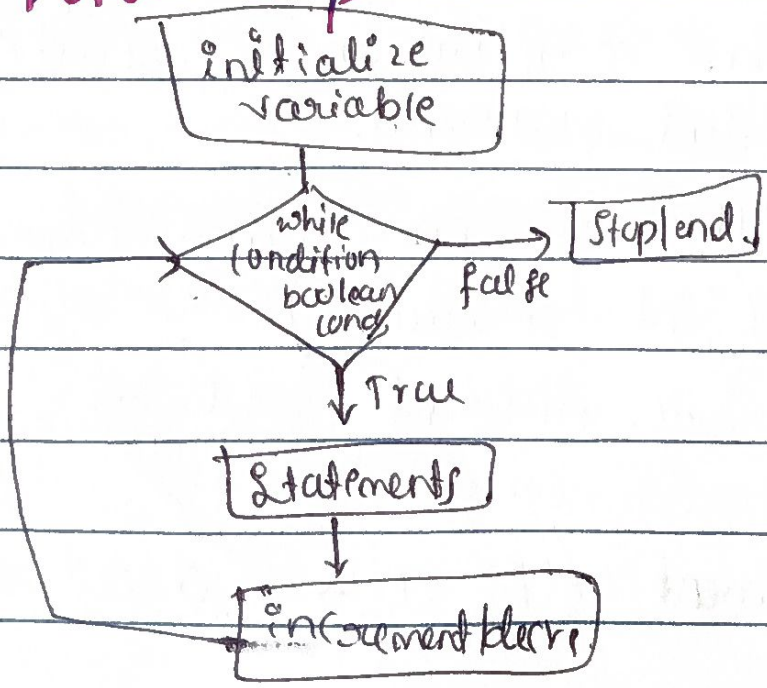
2) range(Start, Stop) → range.
for i in range(11,16):
:
Print(i)
print(i)
o/p — 11 12 13 14 15.
o/p — 11 12 13 14 15.

Ex: for i in range(5):
Print("kgs")
o/p → kgs kgs kgs kgs kgs.

3) range (Start, Stop, Step): → logic.

for i in range(2, 11, +2):
Print(i)
→ 2 4 6 8 10

## ② While loop




Syntax:
initialize variable
while (bool-cond):
:
logic
:
increment/decrement
# remaining Statements

```
(x: i=0
    while (i ≤ 4):
        print("Python")
        i = i+1
    print(i)
```

Python Python Python Python Python

Python Python Python Python Python

## * Jumping Statements

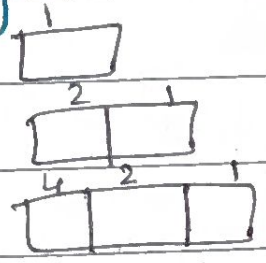
- Break → for i in range(5):
  if i==3:
  break
  print(i)
  o/p = 0 1 2
- Continue
- Pass.

for i in range:
  if i==3:
  continue
  print(i)
o/p — 0 1 2 4.

for i in range(5):
  if i==3:
  pass
  print(i)
→ o/p — 0 1 2 3 4.

ignored by compiler.

## Bit & Byte




| | | |
|---|---|---|
| 1 bit | 0 or 1 | |
| 2 bit | 00 01 10 11 | 0 to $2^2-1$ → 0 to 3. |
| 3 bit | 000 001 010 ... 1111 | 0 to $2^3-1$ <br> 0 to 8-1 → 0 to 7. |
| 4 bit | 0 to $2^4-1$ → | 0 to 15 |
| 5 bit | 0 to $2^5-1$ → | |
| 8 bit | 0 to $2^8-1$. | |

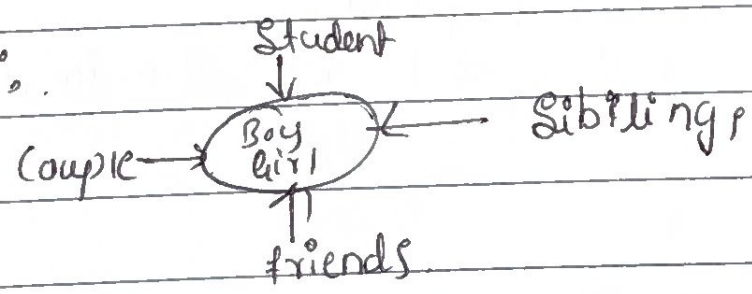
8 bits = 1 byte
Int = 2 byte.
2 byte = 16 bits

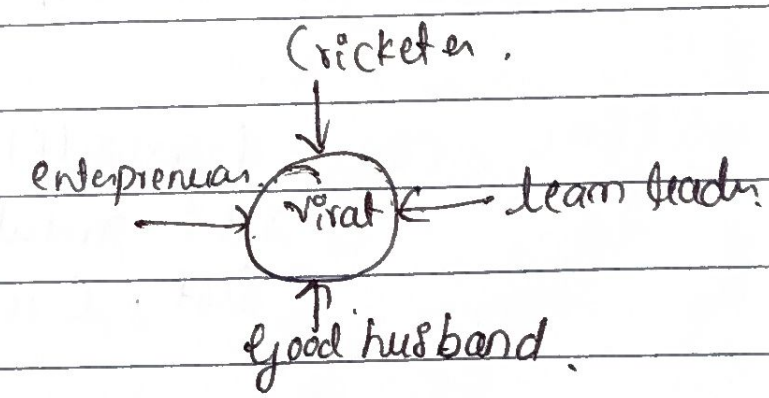
30/07/2026

## Python: 3D

OOPS - Object oriented Program System
- Point of views
- Perspective
- alligment.

ex:




- define a word is object or not
- define type of object
- classify what it has & what it does.

# Class:

Laptop

has — motherboard, Screen, display, key, cost
does — } behaviour & method.

Properties or variable or attributes.

instance of class
(n)
object capacity
House
n

Site → contracter → House n

Planning / blueprint...n.

object.

— use one blueprint we can built so many same.

class Cat:

1 tab space.

first letter uppercase.

Dog

has — breed, color, gender, cost
does — bark, sleep, drink } method.

variable

Property & behaviour belongs to object.

widely used Python.

high cohensional Program.

class Dog:

```
breed = "lab"
color = "black"
gender = "male"
cost = 1300
def bark():
    Print("Dog is bark")
def sleep():
    Print("Dog is sleep")
```

Property & behaviour not belong to object

Low cohensional Program.

## Steps:

- The PVM allocate the sperate block of memory with an memory address.
- A PVM start searching for __init__ constructor once it found allocate the variable and their values & methods in memory.
- The address of an object stored in refference variable.

Ex.

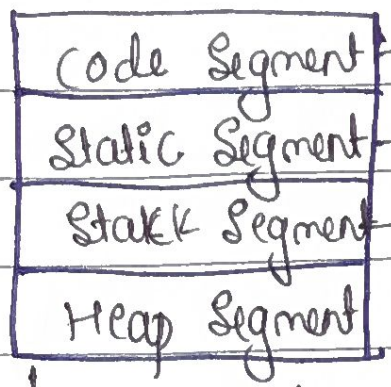
```
class Fan(self):
    self.ratate
    self.cost
```

define keyword but self is standard

* Variable datatypes.

31/07/2026.

# Organization of RAM




Code Segment → Stored whole code.

Static Segment → Stored Static variable.

Stakk Segment → local variable [reference variable / (erase record. after use.) activation record.

Heap Segment → Stored only the object / instance method / instance variable

```
Class Student :
    def __init__(Self):
        Self.name = "Raju"
        Self.age = 23.
        Self.division = "A"
    def write (Self):
        Print ("Raju is writing")
    def read (Self):
        Print ("Raju is reading")
```

Instance variable.

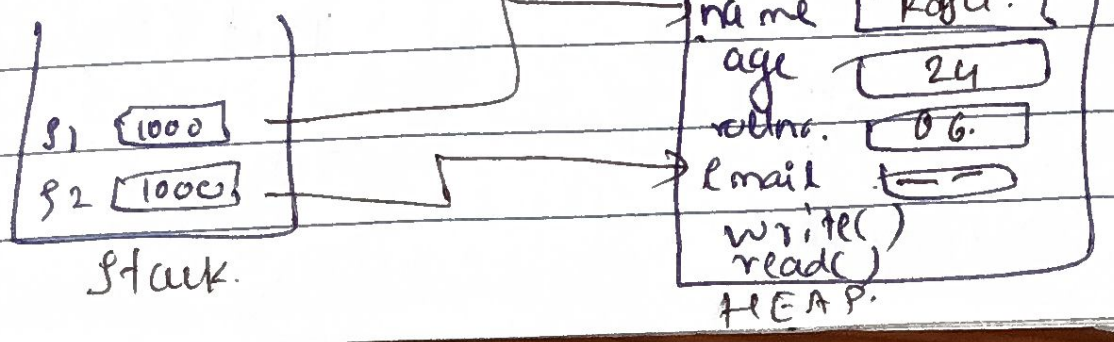
Instance method.

```
S1 = Student()
Print (S1.name)  → Raju
Print (S1.age)   → 23.
S2 = S1.
Print (S1)  → address of an object
Print (S2)  → address of an object
S1.email = "Raju@gmail.com"
Print (S1.email) → "Raju@gmail.com"
S2.age = 24
Print (S2.age) → 24
```

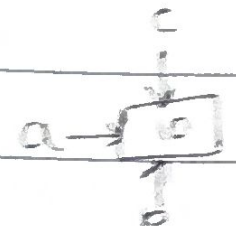
reference variable.

local variable.

o/p



Shallow copy




a=10
b=10
c=10
Print(a) → 10
Print(b) → 10
Print(c) → 10
Print(id(a)) → address
Print(id(a)) → address
Print(a is b) → True
Print(c is b) → True
Print(a is d) → False.

Everything address is changed

→ magic method
__init__

Self → Store every address.

## * Methods in Python.

- instance method
- Static method
- Class method.

### ▷ instance method.

- No Parameter No return value.

```
class Calculator:
    def __init__(Self):
        Self.brand = "casio"
        Self.cost = 200
    def add(Self):
        a=10
        b=20
        c=a+b
        Print(c)
c1 = Calculator.
Print (C1.brand) → casio
Print (C1.cost) → 200
C1.add()  ⟶ 30
```

- With Parameter No return

```
def add(Self.a.b):
    c = a+b
    Print C.
```

Parameter.

```
C1=Calutor.
Print (C1.brand)
C1.add(100.200) → 300
```

- No Parameter with return value.

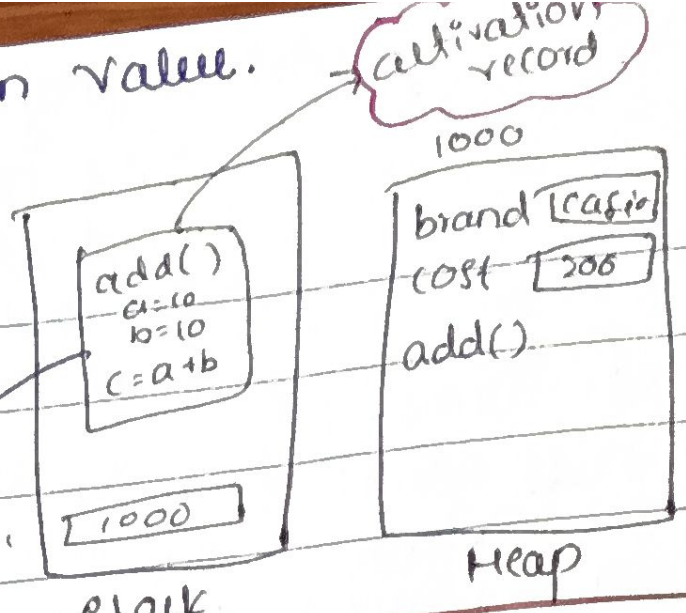
```
Class Calulator:
    def __init__(Self):
        Self.brand = "casio"
        Self.cost = 200
    def add(Self):
        a=10
        b=20
        C=a+b
        return C
C1 = Calulator
Print (C1.brand) → casio
Print (C1.cost) → 200
res = C1.add()
Print (res)
```

Print (

return C

- with Parameter with return value.

```
deff  add (self,a,b):
        c=a+b.
        return c.
c1. calculator.
Print (c1. cost)  ⟶ 200
res = c1.add (100.200)
Print (res)  ⟶ 300
```




automatically dissappear. bc2 Stack having limited memory access

04/08/2026

Declare Variable:

class Demo:

→ initialization.
→ constructor.

```
def __init__(self):
    self.x = 10
    self.y = 20
def display(self):
    Print (self.x).
    Print (self.y)
    self.z = 30
    Print (self.z).
d1 = Demo()
Print (d1.x)
Print (d1.y)
d1.display()
Print (d1.z)  ⟶ it will show ⇒ 30.
Print (d1.z)
```

Here we Print (d1.z) it will show error bc2 it will not store in heap segment first display the variable x, y, z then z is stored in memory

→ Various ways of declare & accessing the instance Variable.

```
class Demo():
    def __init__(self):
        self.x = 10
        self.y = 20
```

```
def display(Self):
    Print (Self.x)
    Print (Self.y)
    Self.z = 30
    Print (Self.z)

d1 = Demo()
Print (d1.x) → 10
Print (d1.y) → 20
d1.display() → 10 20 30
Print (d1.z) → 30
d1.a = 40
Print (d1.a) → 40
```

05/09/26

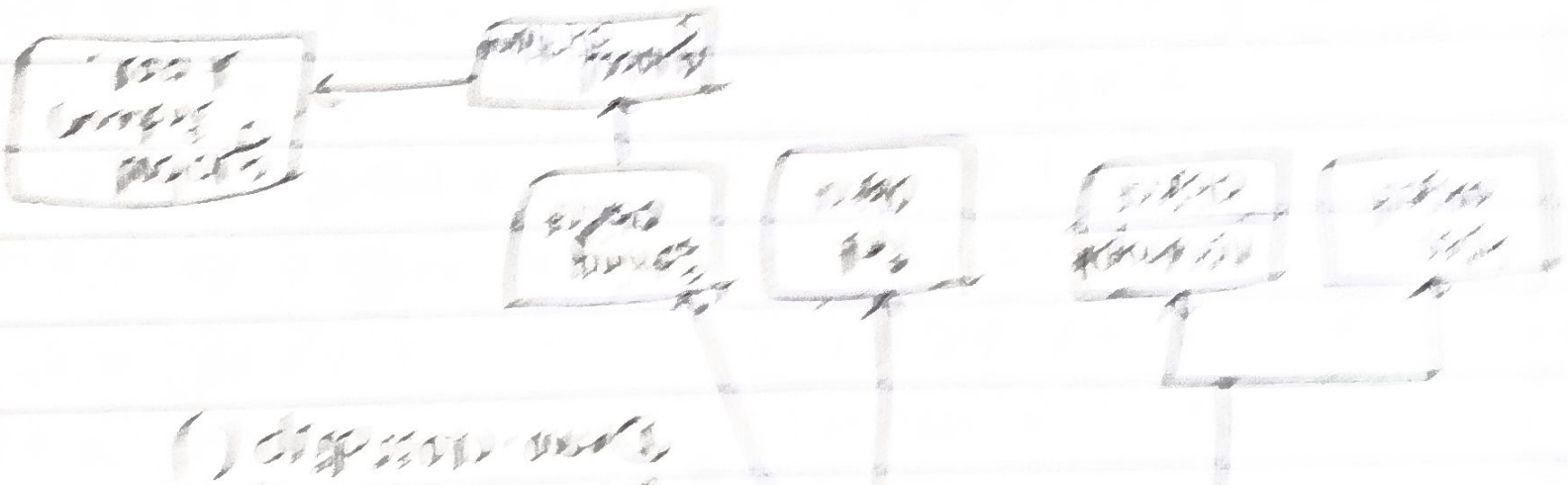
## Applications Of Static Variable:

```
class Farmer :                                  Here we use
    r = 2.5  ← Static variable.                 Static variable
    def __init__(Self.P,T):                     then we used as
        Self.P = P   } Instance variable        Farmer.r
        Self.T = T
    def calSi(Self):
        SI = (Self.P * Self.T * Farmer.r)/100
        Print (SI)

f1 = Farmer (10,000, 2)   ← Instance variable
f1.calSi() ---- 5000
f2 = Farmer (20,000, 4)
f2.calSi() → 20,000
```

② 

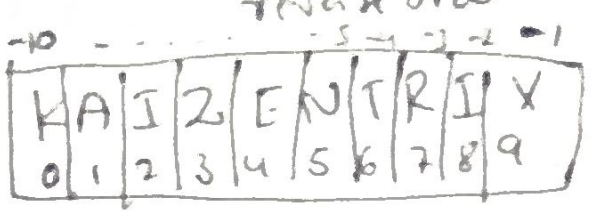
```
class Demo :                                    → Static variable
    a = 10
    def __init__(Self):
        Self.b = 20   } → instance variable
        Self.c = 30
    def display(Self): → instance method.
        d = 40   ← local variable
        Print (d)
```



- **Indexing:**

Str = "kaizentrix"

Space is also one block




Ex: Str = "KAIZENTRIX"

Print(Str) — KAIZENTRIX

Print (Str[3]) — Z

Print (Str[5]) — N

Print (Str[-1]) — X

Print (Str[-4]) — T

Print (len(Str)) — 10

↓ length of String

## Airthmatic operation of String

Str = "jaanu"

Str2 = "Pooja"

Str3 = Str + +Str2 → concatination

Print (Str3) PoojajaanuPooja

Str3 = Str + " " +Str2.

Print (Str3) Jaanu Pooja

unSupported operand

# Str4 = Str - Str3

# Print (Str4)

# Str5 = Str * Str2

# Print (Str5)

Str6 = Str * 2

Print (Str6) jaanujaanu

Str7 = Str2 * 3.

Print (Str7) PoojaPoojaPooja

# Str8 = Str / Str2

# Print (Str8)

- **Slicing**

Print (Str [3:7:+1])

start, stop, step

Print (Str [3: ])

→ it will print till end

Print (Str[:])

it will Print complete string

Print (Str[1:6:+2]) → a2n

alternate letter print.

Print (Str [8:3:+1])

no output

Print (Str [8:3:-1])

irtne

Print (Str [-8:-2:+1])

IZENTR

Print (Str [-8:-4:-1])

no output

Print (Str[::-1])

XIRTNEZIAK

Print (Str[-8:-4:0])

error

# String interning

```
Str1 = "jaanu"
Str2 = "pooja"
Str3 = "Darshan"
Print(Str1)
Print (Str2)
Print (Str3)
Str4 = "jaanu"
Str5 = "pooja"
Str6 = "darshan"
Print (id(Str1)) - 1356
Print (id(Str4)) - 1356
Print (id (Str2)) - 4656
Print (id (Str5)) - 4656
Print (id (Str3)) - 4848
Print (id (Str6)) - 4848
Print (Str3 is Str6) True
Print (Str1 is Str6) false.
```

# ASCII

American Standard code for information interchange

| | | |
|---|---|---|
| a - 97 | A - 65 | 0 - 48 |
| b - 48 | B - 66 | 1 - 49 |
| : | : | 2 |
| : | : | : |
| z - 122 | Z - 90 | 9 - 57 |

convert uppercase to lowercase +32

convert lowercase to uppercase. -32.

char(value) converts = ASCII → ord.

ord.

ASCII = char → Chr.

Ex: char = input ("Enter a Single:") #A
data = ord (char)
Print (data) #65

ASCII = int(input("Enter a value:")) 69
data = Chr(Ascii)
Print (data) #E.

# Function:

User defined  Predefined

1) User defined

a) No parameter no return

```
def add():
    a = 10
    b = 20
    c = a+b
    Print(c)
add()  ⇝ 30
```

b) No parameter with return

```
def add():
    a = 10
    b = 20
    c = a+b
    return c
r1 = add()
print(r1)  ⇝ 30
```

c) with Parameter No return

```
def add(a,b):
    c = a+b
    print c
x = 10
y = 20
add(x,y)  ⇝ 30
```

d) with parameter with return

```
def add(a,b):
    c = a+b
    return c
x = 10
y = 20
r1 = add(x,y)
print(r1)  ⇝ 30.
```

diff b/w function

method:

function - it can be called directly by its name.

- block of code that performs specific task.

Method: it is called using an object or class

- is function that belongs to an object.

higher order function

```
class Demo:
    def disp(self, x=11, y=22, z=33):
        Print(x)
        Print(y)
        Print(z)

d1 = Demo()
a = 10
b = 20
c = 30
d1.disp(a,b,c) => 10
                  20
                  30.
d1.disp() # Error -> positional arguments is missing.
          when give value in def
d1.disp(a,b)  # error - one positional argument missing
d1.disp()  11, 22, 33
d1.disp(a,b,c)  10, 20, 30
d1.disp(a,b)  10, 20, 33
d1.disp(b,c)  20, 30, 33
d1.disp(y=b, z=c)  11, 20, 30
d1.disp(x=a, y=b, z=c)  10, 20, 30.
```

default arguments.

1st 2nd 3rd

invoking

17/08 Even number

without using inbuilt. —code

With using inbuilt function → filter- is keywort it is process to extract the elements in list using conditional Statement 1 particular.

Syntax: → To get best one

filter (func_name, list of elements)

```
def even(num):
    If num%2==0
        return True
    else
        return false.

l=[]
i=0
while i<=4:
    n=int(input("Enter number"))
    l.insert(i,n)
    i=i+1
Print(l)
i=0
while i<=4:
    data=l[i]
    choice= even(data)
    if choice == True
        Print(l[i])
    i=i+1.
```

```
K=list(filter(even,l))
Print(K)
```

* Square of number

```
def Square(n):
    Sq=n*n
    return Sq
n=int(input("Enter number"))
res=Square(n)
Print(res
```

anonymous func

using lambda

```
K=lambda n: n*n
data=K(5)
Print(data)
```

that can take any no. of arguments but can only contain single expression

```
K= lambda
a,b = a*b
data=K(a,b)
Print(data)
```

* using filter implement lambda function,

Syntax:

filter(lambda arguments: expression, list of elements)

ex:

```
l=[1,2,3,4,5]
K= list (filter(lambda n: n%2==0, l))
Print(K)
```

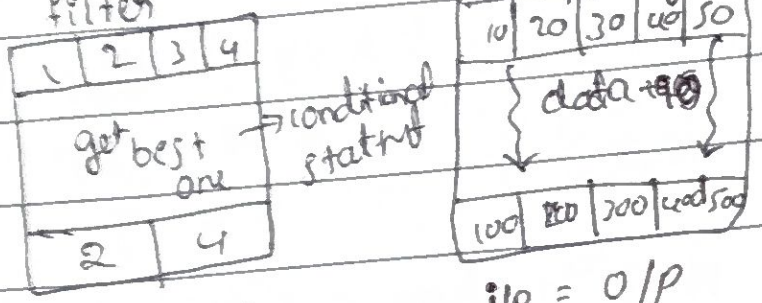
o/p = [2,4]

18/08/26

# Mapping:

Syntax: map(func-name, list of elements)

→ add the data and give new list.




Map + list

Ex: l = [1, 2, 3, 4]

add = data + 5

print(l)

k = list(map(operation, l)) ← func it

print(k)

→ o/p = [6, 7, 8, 9]

## Map + lambda

map(func-name, list of elements)

lambda arguments: expression

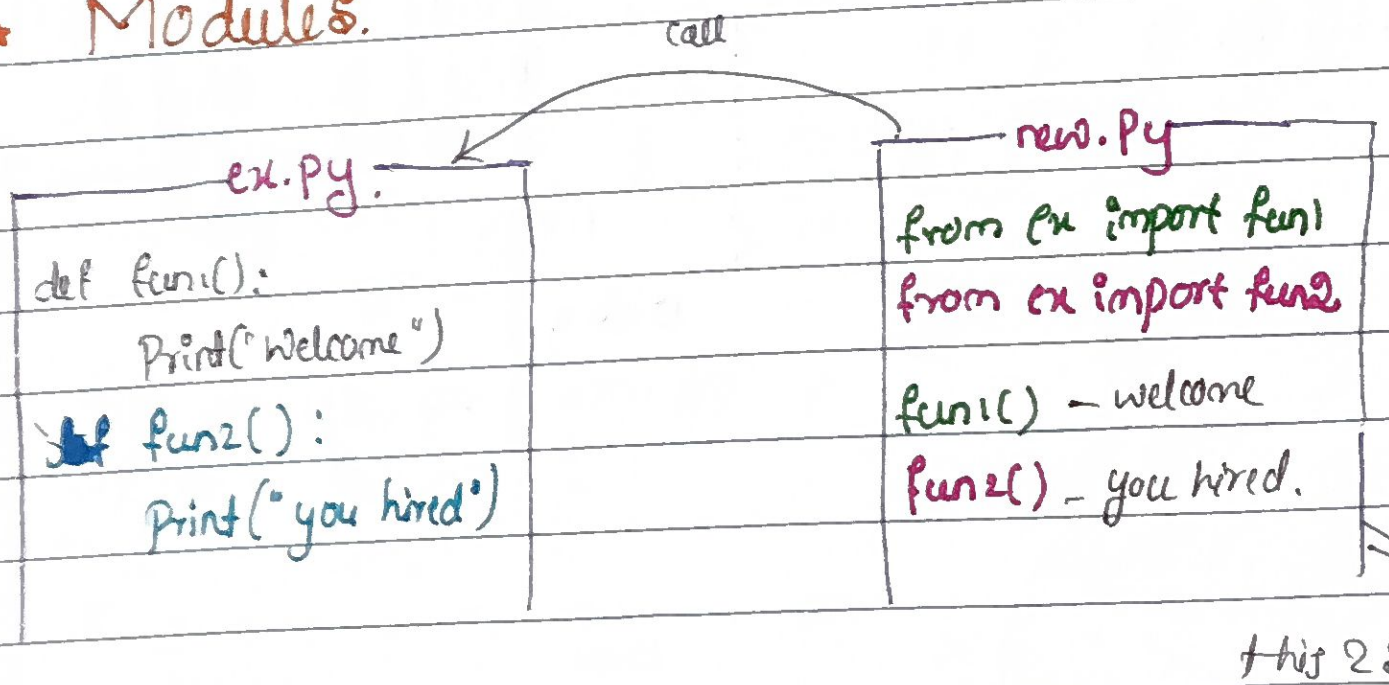
Syntax:

⇒ map(lambda arguments : expression, list)

Ex: k = list(map(lambda data: data+10, l))

print(k)

## * Modules.




from ex import * or from ex import fun1, fun2

**Pgm1.Py**

```
def fun1():
    Print("inside fun1 in pgm1")
def fun2():
    Print("inside fun2 in pgm1")
```

updated import function is executed

**new.Py**

```
from pgm1 import fun1, fun2
from pgm2 import fun1, fun2
fun1() — inside fun1 in pgm2
fun2() — inside fun2 in pgm2
fun1() — inside fun1 in pgm2
fun2() — inside fun2 in pgm2
```

**Pgm2.Py**

```
def fun1():
    Print("inside fun1 in pgm2")
def fun2():
    Print("inside fun2 in pgm2")
```

How to overcome this problem:

**new.Py**

```
import Program1   Pgm1
import Program    Pgm2
Pgm1.fun1()  ~ inside fun1 in Pgm1
Pgm1.fun2()  ~ inside fun2 " "
Pgm2.fun1()  ~ " fun1 " "Pgm2
Pgm2.fun2()  → " fun2 " "
```

## Nested function

Syntax:

```
def outer func_name():
    ==========
    def inner func_name():
        ==========
    def inner func_name()
outer func_name()
```

2000

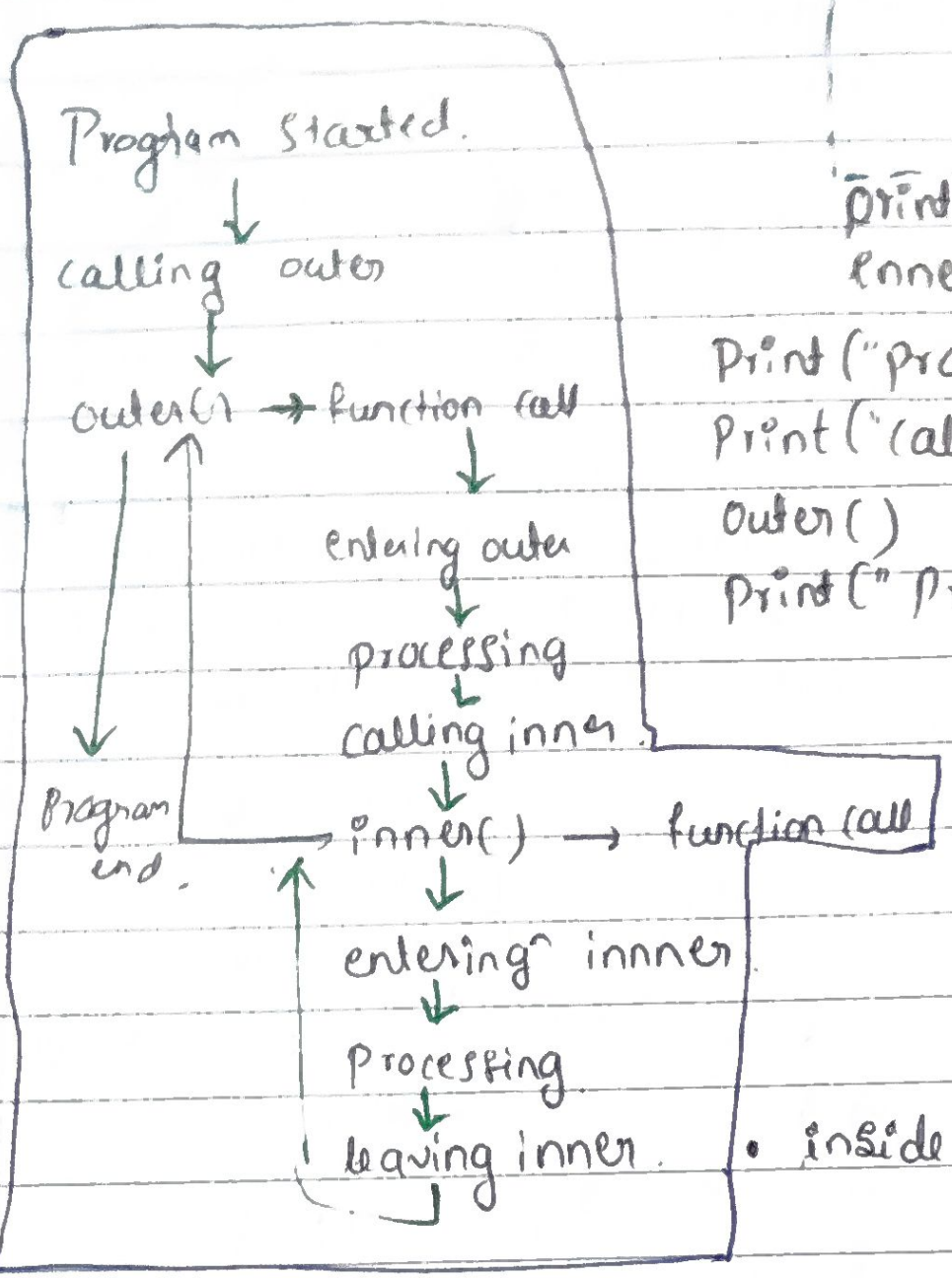
```
def outer():
    Print("Entering outer")
    1000
    def inner():
        Print("Entering inner")
        Print("Processing")
        Print("leaving innner")
    inner()
Outer()
```

inner() → it will ultimately show an error.

# Control flow of nested function.

19/01/2026

```
def outer():
    Print ("Entering outer")
    Print ("processing)
    def inner():
        Print ("Entering inner")
        Print ("Processing")
        print ("leaving inner")
    Print ("calling inner")
    Inner()
Print ("Program started")
Print ("calling outer")
Outer()
Print ("Program end)
```




if suppose we can declare variable in inside outer function, is called "non local var"

- inside inner function → "local variable"

Here declare variable is called → global variable. (then Print(a))

```
def fun1():
    a = 10
    print(a)
    def fun2():
        Print (a)
        b = 20
        Print (b)
        Print (a)
print(a)
fun2()
#Print(b)
#Print(a)
fun1()
#Print(a)
```

non local variable

local variable

not defined

10
10
10
20
10

not define. b(2 Activation record will be erased.

without calling function, don't print a. b(2 a is not defined.

→ Activation record erased.

```
def fun1():
    a = 10
    Print(a)

def fun2():
    non local a
    a = 100
    Print(a)
    b = 20
    print(b)
    Print(a)
Print(a)
fun2()
Print(a)
fun1()
```

how to call inner function in outer fun. outside.

non local a → it convert local to global variable.

```
def outer():
    Print("Entering")
    def inner():
        Print("entering")
        Print("Bcoes")
    return inner

ref = outer()
Print(ref) → Print Add
ref() → function call
```

To store inner fun address.

Closure - The Processing of invoke the inner function is outside outer function.

20/08/2026.

## Decorator : (@)

```
def great():
    Print("Hi EveryOne")

def outer(Print1):
    Print("Entering outer")
    def inner():
        Print("Entering inner")
        ref = Print1
        ref()
        Print("leave inner")
    return inner

ptr1 = great
ptr2 = outer(ptr1)
ptr2()
```

3000
1000
2000

⇒ o/p :

- Hi everyone
- entering outer
- entering inner
- hi every one
- Leaving inner

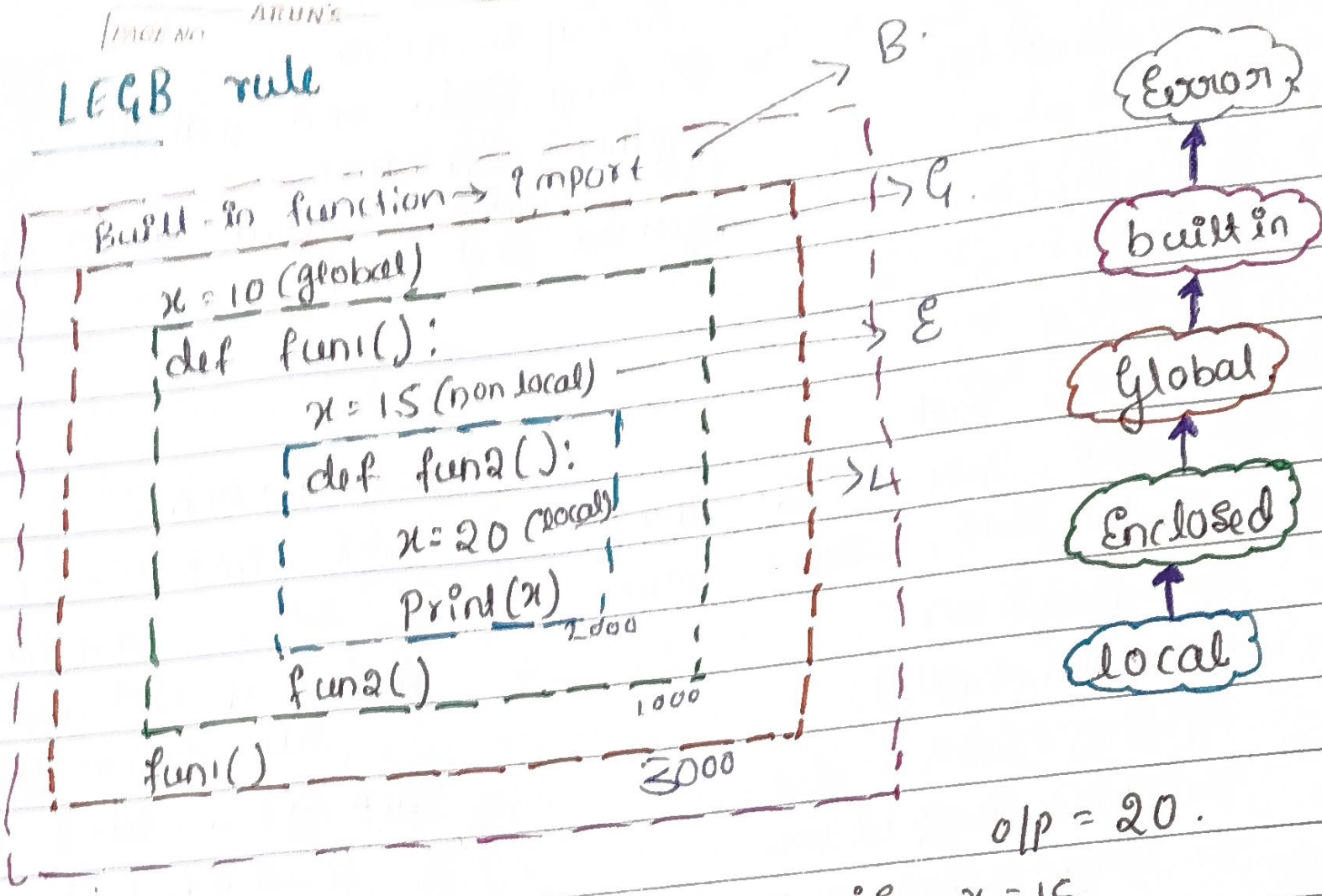
Decorator

```
@outer
def great():
    Print("Hello")
great()
```

Note.

```
great = outer(great)
great()
```

# LEGB rule




o/p = 20.

incase I comment # x = 20 o/p is x = 15

#x=20 & x=15 o/p is x = 10

# x=20 , x=15, & x = 20 o/p is is NOD.

24/08/2026